

Autonomous Signals: A New Ontology for AI Agents

*The ButterClaw Containment Architecture and the Conceptual Shift
from Language Models to Signal Models*

Bc77

ButterClaw Tech

ORCID 0009-0001-2162-1397

research@butterclaw.tech • October 2026

Abstract

The field of artificial intelligence has long operated under a vocabulary inherited from its earliest paradigms: “language models,” “chatbots,” “agents,” “assistants.” This paper argues that these terms are not merely imprecise—they are *epistemically dangerous*. When the ontological frame does not match a system’s actual behavior, safety guarantees built on that frame become structurally unsound.

We propose a principled reframing: large language models are more accurately described as *signal models*—systems that transduce, transform, and emit structured semantic signals—and autonomous AI agents are more precisely understood as *autonomous signal loops*, self-routing information processes that accumulate context, initiate actions, and resolve without continuous human mediation. This reframing surfaces a new safety surface defined by signal boundaries, loop depth, side-effect potential, and termination conditions rather than token probabilities and prompt structures.

To address this surface, we introduce **ButterClaw**—a containment-first reference architecture for safe autonomous signal deployment, now at implementation version **v0.9.2**. ButterClaw defines six architectural layers, a five-hemisphere reasoning model that separates decisive, skeptical, speculative, self-improving, and fleet-level cognition into independently scheduled LLM passes, a named policy evaluation runtime (DRIFT) spanning three signal-lifecycle scopes, a three-tier kill switch model (Signal Pause, Loop Drain, Hard Termination) with a concrete credential-destruction mechanism (Gibson), four containment primitives (Boundary, Depth, Consent, Observability) mapped onto the runtime’s enforced invariants, and a formal threat model for adversarial signal injection.

Licensing: The ButterClaw runtime architecture and source code are licensed under Apache-2.0. This whitepaper is licensed under a Creative Commons Attribution 4.0 International License (CC-BY-4.0).

Keywords: autonomous signals, signal models, agentic AI, AI safety, containment architecture, kill switch, signal loops, ButterClaw

Primary Classification: cs.AI **Cross-list:** cs.SE, cs.SY

Contents

1	Introduction: The Naming Problem in AI	3
1.1	Why Terminology Shapes Safety	3
1.2	Paper Roadmap	3
2	From Language Models to Signal Models	3
2.1	What a Language Model Actually Does	3
2.2	Redefining the Unit of Analysis: The Signal	3
2.3	Framing Comparison	4

2.4	Implications for Design and Evaluation	4
3	Autonomous Signals	4
3.1	Definition	4
3.2	Anatomy	4
3.3	Properties	4
3.4	Taxonomy	4
3.5	Signal Lifecycle Diagram	5
4	Signal Loops: The Architecture of Agentic Behavior	5
4.1	Loop Typology	5
4.2	The Stability Problem	5
4.3	Loop Depth Bounding	6
4.4	Inter-Agent Signal Propagation	6
5	The ButterClaw Architecture	6
5.1	Core Design Principles	6
5.2	Architectural Layers	6
5.3	The Context Membrane	6
5.4	Five-Hemisphere Reasoning	8
5.5	The DRIFT Policy Engine	9
5.6	The Memory Substrate and Offline Consolidation	9
5.7	Fleet Detection Stack	11
5.8	Comparison with Conventional Frameworks	12
6	Kill Switch Architecture	12
6.1	Why Kill Switches Fail	12
6.2	The Three-Tier Kill Switch Model	12
6.3	Kill Switch Triggers	13
6.4	The Orphan Signal Problem	13
6.5	Recovery and Re-Entry Protocols	13
6.6	Gibson: Concrete Credential Wipe Implementation	13
7	The Signal Containment Model	14
7.1	Containment as a First-Class Design Primitive	14
7.2	The Four Containment Primitives	14
7.3	Formal Properties	14
7.4	Containment Threat Matrix	14
7.5	Implementation Mapping	14
8	Related Work	16
8.1	Agent Frameworks	16
8.2	Prompt Injection	16
8.3	Kill Switch Research	16
9	Software Artifact and Reproducibility	16
9.1	Repository	17
9.2	The butterclaw-sentinel SDK	17
9.3	Citation	17
9.4	Version Binding	17
9.5	Practical Deployment	17
9.6	Interoperability	17
10	Open Problems	18
11	Conclusion	18

1. Introduction: The Naming Problem in AI

Language is infrastructure. The terms a technical community uses to describe its artifacts do not merely label—they constrain. They determine which properties get measured, which failure modes get anticipated, and which safety primitives get designed. The field of artificial intelligence has accumulated a vocabulary increasingly mismatched with the systems it seeks to describe. This mismatch is not a cosmetic inconvenience. It is a structural risk.

The canonical AI vocabulary—“language model,” “chatbot,” “assistant,” “agent”—was assembled when these systems were narrow, stateless, and principally text-bound. Contemporary frontier models can initiate tool calls, traverse multi-step reasoning chains, maintain context across thousands of turns, spawn subagents, write and execute code, modify files, dispatch emails, and transact with external APIs—all within a single inference episode. Yet we still call them “language models” and frame their safety properties in terms of content policy and output filtering.

Imprecise ontology produces imprecise safety guarantees. The names we give AI systems determine the failure modes we protect against—and therefore the failure modes we leave unprotected.

1.1 Why Terminology Shapes Safety

AI agents modeled as “language models”—token-prediction engines—receive safety engineering appropriate for text generation: output filtering, probability thresholds, toxicity classifiers. They do not receive safety engineering appropriate for autonomous signal loops: loop depth bounding, context membrane enforcement, federated kill switches, orphan signal detection. The gap between what is needed and what is provided is the direct product of the nomenclature gap.

1.2 Paper Roadmap

This paper proposes a systematic reframing. Section 2 develops the signal model concept. Section 3 defines autonomous signals. Section 4 introduces signal loop architecture. Section 5 presents ButterClaw, including its five-hemisphere reasoning model, its DRIFT policy engine, its memory substrate, and its fleet detection stack as implemented in v0.9.2. Sections 6 and 7 address kill switch design (including the Gibson credential-destruction mechanism) and containment modeling, mapped onto the runtime’s enforced invariants. Section 8 situates ButterClaw against conventional agent frameworks and prior kill-switch and prompt-injection research. Section 9 describes the released software artifact, its SDK, and how to cite this work. Section 10 treats open problems. Section 11 concludes.

2. From Language Models to Signal Models

2.1 What a Language Model Actually Does

At the computational substrate level, a “language model” computes a probability distribution over the next token $x_{n+1} \in \Sigma$ given an input sequence $\mathbf{x} = (x_1, \dots, x_n)$. This describes the *mechanism*, not the *function*. The function is the transformation of an input semantic structure into an output semantic structure with semantic payload, implicit commitments, and potential real-world consequences.

“Token prediction is the mechanism of signal transduction, not the function of signal transformation. Describing a large language model purely as a next-token predictor is like describing a telephone as a membrane vibration device.”

2.2 Redefining the Unit of Analysis: The Signal

Definition 2.1 — Signal

A **signal** is a structured unit of information characterized by: (1) a *semantic payload*—propositional content with interpretable meaning; (2) a *temporal context*—a timestamp, session state, or causal ordering; (3) a *transformative intent*—the change to be effected in the receiving system; and (4) a *provenance record*—a traceable origin identifying the initiating agent or process.

2.3 Framing Comparison

Table 1: Language Model Framing vs. Signal Model Framing

Dimension	Language Model	Signal Model
Primary Unit	Token / sequence	Signal (payload + provenance)
Input Type	Text prompt	Structured signal with intent & context
Output Type	Text completion	Transformed signal w/ side-effect commitment
Temporal Awareness	Implicit (context window)	Explicit (timestamp, causal ordering)
Safety Surface	Content (toxicity, accuracy)	Boundaries, loop depth, consent, observability
Containment Req.	Filtering, classifiers	Isolation, loop bounding, kill switch, orphan detection
Failure Vocabulary	Hallucination, jailbreak	Loop divergence, orphan propagation, consent bypass

2.4 Implications for Design and Evaluation

Evaluation suites must expand to include signal-level metrics: side-effect potential per inference call, loop initiation rate, external state modification surface, consent solicitation frequency. Architecture decisions must treat signal isolation as a first-class design constraint. Safety engineering must be defined at signal boundaries, depths, and consent surfaces—not at output content alone.

3. Autonomous Signals

3.1 Definition

Definition 3.1 — Autonomous Signal

An **autonomous signal** is a signal that initiates, routes, and resolves itself without requiring *continuous* human mediation at each step of its lifecycle. It carries sufficient context, routing logic, and resolution criteria to propagate through a computational environment and effect state changes independently, subject only to the constraints of its containing architecture.

3.2 Anatomy

An autonomous signal has five structural components: **Origin** (provenance record, preserved throughout lifecycle); **Payload** (semantic content); **Routing Logic** (next-hop specification, potentially model-generated); **Resolution Criteria** (conditions for signal closure); **Termination Condition** (conditions for cessation regardless of resolution state).

3.3 Properties

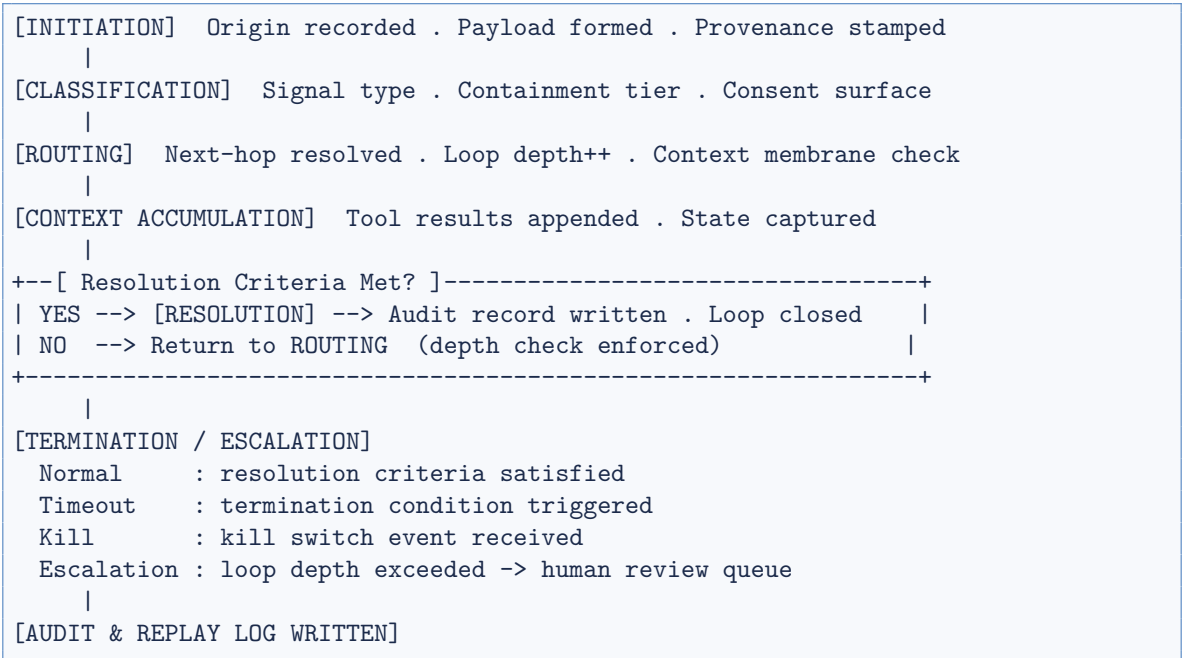
Persistence: State maintained across multiple steps. **Self-Routing:** Next destination determined at least partially from payload + environment. **Context Accumulation:** Tool outputs and environmental state appended at each hop. **Side-Effect Potential:** External state (files, APIs, communications) may be modified.

3.4 Taxonomy

Reactive: Triggered by external event; bounded single-path response. **Proactive:** System-initiated on schedule or condition; higher autonomy. **Recursive:** Spawns child signals; creates tree/graph propagation structures. **Federated:** Crosses organizational or system trust boundaries.

3.5 Signal Lifecycle Diagram

Listing 1: Autonomous Signal Lifecycle in ButterClaw



4. Signal Loops: The Architecture of Agentic Behavior

Definition 4.1 — Signal Loop

A **signal loop** is a closed-loop computational structure in which the output of a signal transformation re-enters the system as input to a subsequent transformation, without requiring external human re-initiation between cycles. Signal loops are the fundamental architectural unit of agentic behavior.

4.1 Loop Typology

Table 2: Signal Loop Typology

Type	Description	Risk	Containment Priority
Open Loop	Unidirectional; no feedback	Low	Audit logging; output validation
Closed Loop	Output re-enters as input; criterion governs exit	Medium	Depth limit; timeout; resolution validation
Recursive Loop	Spawns child loops; tree/graph structure	High	Stack depth accounting; child inherits parent constraints
Federated Loop	Hops cross trust/system boundary	Very High	Cross-domain trust; federated kill switch; provenance attestation

4.2 The Stability Problem

For signal loops driven by large language models, the routing function is a high-dimensional nonlinear transformation learned from data. Formal stability proofs are not currently tractable. This makes empirical loop monitoring and depth-bounding essential safety primitives—in the absence of provable convergence, hard limits must be enforced architecturally.

4.3 Loop Depth Bounding

Loop depth is the number of autonomous signal transformations since the initiating signal was created without human re-initiation. *Maximum permitted depth* is a hard architectural configuration parameter—the ceiling beyond which no signal may propagate regardless of routing logic. Correct configuration is context-dependent: a code generation agent may safely operate at depth 10; an autonomous financial transaction agent may require depth 2 with mandatory consent gates at every level. In the reference implementation this ceiling is enforced concretely by the **ChainExecutor**, which executes a maximum of ten steps per chain under a sixty-second total timeout (Invariant I-07; section 7.5).

4.4 Inter-Agent Signal Propagation

In multi-agent systems, containment requires graph-level properties: total propagation path depth must be bounded; trust attestations must be verified at each inter-agent boundary; kill switch events must propagate bidirectionally through the graph.

5. The ButterClaw Architecture

Definition 5.1 — ButterClaw

ButterClaw is a six-layer reference architecture for autonomous signal deployment. Its central design commitment is that containment is a first-class architectural primitive. Every signal that enters a ButterClaw-compliant system is classified, bounded, monitored, and subject to controlled termination from the moment of ingestion.

5.1 Core Design Principles

1. **Signal Isolation:** Signals isolated by default; sharing requires policy authorization.
2. **Loop Bounding:** Hard depth limit enforced architecturally, not at application layer.
3. **Consent Surfaces:** External state modification requires verified consent traversal.
4. **Observable State:** No “dark” signals; all state emitted to monitoring surface in real time.
5. **Graceful Degradation:** Kill events preserve state, drain in-flight signals, lock forensic records.

5.2 Architectural Layers

Figure 1 illustrates the complete data flow through all six layers.

Layer 0 — Signal Ingestion & Classification: Single ingestion endpoint; UUID stamping; provenance; containment tier assignment. Unclassifiable signals quarantined.

Layer 1 — Context Membrane: Stateful semantic boundary. Maintains signal integrity via cryptographic payload authentication. Enforces per-signal context scoping to prevent contamination. Primary defense against prompt injection into signal payloads.

Layer 2 — Routing Engine: Policy-graph-governed dispatch. Permitted routing paths, mandatory consent checkpoints, and dead-end rules for depth-exceeded signals. As of v0.9.0–v0.9.2 this layer’s decision surface is implemented concretely by the five reasoning hemispheres (section 5.4) and arbitrated by the DRIFT policy engine (section 5.5).

Layer 3 — Loop Monitor: Real-time registry of active loops, depths, initiation times, and side-effect logs. Evaluates invariants at every depth increment. Issues kill commands on threshold violations.

Layer 4 — Resolution & Termination Controller (RTC): Processes resolution events and termination events. Implements the three-tier kill switch. Ensures no signal continues after a termination event for its signal tree.

Layer 5 — Audit & Replay Layer: Append-only, cryptographically chained audit log. Full replay capability for forensic analysis. Availability is a *prerequisite* for signal execution, not an optional monitoring enhancement.

5.3 The Context Membrane

The context membrane operates at the semantic layer, not the network layer. Its core operation is *context scoping*: at each hop, the signal’s available context is exactly the information legitimately accumulated through its lifecycle plus any information explicitly released by routing policy. Its secondary operation is *payload authentication*:

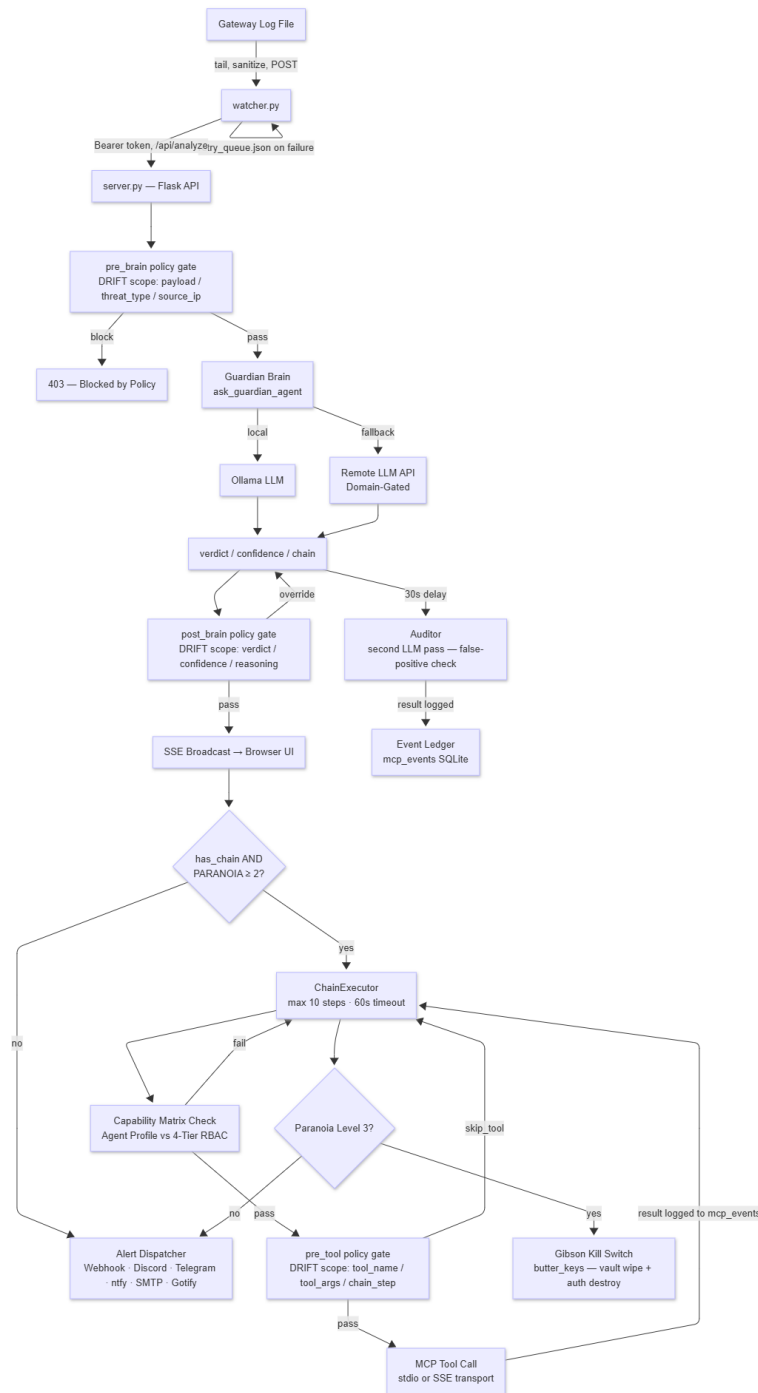


Figure 1: High-level data flow through the ButterClaw runtime. Requests enter via the Watcher Daemon (Layer 0), traverse the **pre_brain** context membrane (Layer 1), are evaluated by the Guardian Brain hemisphere (Layer 2), monitored by the Loop Monitor (Layer 3), executed by the ChainExecutor/RTC (Layer 4), and written to the immutable audit log (Layer 5).

modification between hops without authorization triggers quarantine and Loop Monitor alert. This defends against *context contamination injection*—a class of attack in which a malicious signal attempts to read or write context belonging to another signal.

5.4 Five-Hemisphere Reasoning

Rather than routing every decision through a single LLM call, ButterClaw distributes its reasoning workload across five independent LLM hemisphere instances, each with a distinct mandate, temperature, and activation trigger. This design operationalises the paper’s signal model framing at the implementation level: each hemisphere is itself an autonomous signal with a scoped input domain, a defined output action space, and an activation condition that determines when it fires. No hemisphere can substitute for another; their mandates are disjoint by design. The architecture grew incrementally—a dual-hemisphere design (Guardian Brain, Auditor) in early releases, extended to four hemispheres in v0.8.0 (adding the Dream Weaver and Loop Proposer), and to five in v0.9.0 with the fleet-scoped Fleet Sentinel.

Table 3: The five ButterClaw reasoning hemispheres.

Hemisphere	Temp.	Trigger	Mandate	Output Actions
Guardian Brain	0.3	Every request clearing <code>pre_brain</code>	Evaluate incoming request and act	Structured verdict (ALLOW / WARN / BLOCK / ESCALATE / SHUTDOWN) with confidence and reasoning
Auditor	0.0	30 s after every CRITICAL verdict	Was the Guardian Brain correct?	<code>FALSE_POSITIVE</code> flag written to the Event Ledger; diagnostic only, never reverses a terminated chain
Dream Weaver	0.7	Idle ≥ 15 minutes	What threat patterns have not been seen?	Unconditional maturation tick; optional synthetic adversarial scenario written to the semantic graph as <code>source="dream", verdict="SIMULATED"</code>
Loop Proposer	0.4	Every 6 hours	How can containment logic improve?	One signature/policy/prompt change proposed, replayed against recent <code>mcp_events</code> , scored, and committed or reverted
Fleet Sentinel	0.5	Correlation/collusion event received, or every 4 hours	Are agents in the fleet colluding?	Fleet-scope verdict (ISOLATED / CORRELATED / COLLUDING / INSUFFICIENT_DATA); escalates through Guardian Brain + DRIFT on COLLUDING

The temperature assignments are deliberate. The Guardian Brain runs at 0.3 — low enough to suppress hallucination on safety-critical decisions, high enough to reason about novel inputs. The Auditor runs at 0.0 (greedy decoding) to maximise reproducibility: given the same context, it must always reach the same conclusion. The Dream Weaver runs at 0.7, the only hemisphere run warm, because scenario synthesis benefits from imaginative range that a decisive verdict cannot afford. The Loop Proposer runs at 0.4, warm enough to consider more than the most obvious fix while still producing a single well-formed proposal. The Fleet Sentinel sits at 0.5, between the Guardian Brain’s decisive 0.3 and the Dream Weaver’s speculative 0.7: fleet-level coordination analysis benefits from more flexibility than single-event evaluation while still needing to produce a structured, actionable verdict. A single LLM call cannot simultaneously optimise for decisive action, skeptical review, speculative imagination, self-improvement, and fleet-level behavioral reasoning—these are five different, often opposing, prompt pressures, and the separation of evaluation (Guardian Brain) from retrospective audit (Auditor) implements a principle

analogous to separation of powers: no single reasoning process can both make and ratify a containment decision. A priority-queued **HemisphereScheduler** arbitrates contention between hemispheres: the Guardian Brain is cap-exempt (priority 0) so it is never starved by background fleet workload, reactive Fleet Sentinel invocations share priority 1 with the Auditor, and proactive/scheduled hemispheres run at priority 2–4.

Two of the five hemispheres—the Dream Weaver and the Loop Proposer—are *offline consolidation* processes: they do not participate in the real-time request pipeline and cannot issue a BLOCK verdict. This separation of online decision-making from offline self-improvement is discussed further in section 5.6.

5.5 The DRIFT Policy Engine

DRIFT (Dynamic Runtime Inference Filter Topology) is the named policy evaluation runtime that enforces the four containment primitives defined in section 7. It operates across three evaluation scopes, each triggered at a different point in the signal lifecycle and each with a distinct set of available context fields and permitted actions.

Table 4: DRIFT evaluation scopes, trigger points, available fields, and valid actions.

Scope	Trigger Point	Available Fields	Valid Actions
pre_brain	Before the Guardian Brain’s LLM call	payload, threat_type, source_ip (plus derived fields: payload_length, has_chain, hour_of_day, day_of_week, agent_id)	allow, block, override_critical, override_benign
post_brain	After the Guardian Brain’s verdict, before tool execution	verdict, confidence, reasoning (plus all pre_brain fields)	allow, block, override_critical, override_benign, require_confidence
pre_tool	Before each individual tool call in a chain	tool_name, tool_args, chain_step (plus all post_brain fields)	allow, block, override_critical, override_benign, skip_tool

Policy rules are expressed as JSON documents evaluated by a safe interpreter. The interpreter exposes exactly 15 operators—arithmetic, comparison, logical, and string operations—and explicitly excludes `eval()`, `exec()`, and any form of dynamic code execution. This constraint is a deliberate trust boundary: policy authors can express arbitrarily complex routing logic without being able to inject executable code into the policy runtime itself. All string operators are case-insensitive (both sides lowercased before comparison). At the **pre_tool** scope specifically, a capability-matrix RBAC bounds check fires *before* signature matching and can return **skip_tool** immediately on violation; signature matching against the Threat Signature Arsenal runs next (**pre_brain** and **pre_tool** only); ordinary policies, sorted ascending by priority (lowest number = highest priority), run last across all three scopes. The Zero-Day Arsenal extends this with a fleet-integrated signature system; each Arsenal entry carries **collusion_role** and **fleet_scope** fields, allowing a single policy entry to match across both individual and fleet-level threat patterns (section 5.7).

Invariant I-05: ALLOW Never Short-Circuits. A critical invariant of the DRIFT evaluator is that an ALLOW action never short-circuits remaining policy rules in the same scope. Every rule in a scope evaluates in sequence regardless of prior ALLOW verdicts; an ALLOW match is logged and evaluation continues. Only **BLOCK**, **override_critical**, **override_benign**, **skip_tool**, or **require_confidence** can short-circuit the pipeline, with the first non-allow match at the lowest priority number winning. This means that a policy set can enforce layered independent checks—consent, rate limiting, depth limiting, and content filtering—without any individual rule being able to “pass” a request past the remaining checks. This invariant is referred to as I-05 in the implementation’s invariant register; see section 7.5 for its formal property mapping.

5.6 The Memory Substrate and Offline Consolidation

The memory substrate implements a three-tier architecture—HOT, WARM, and COLD—that mirrors the biological distinction between working memory, consolidating recall, and crystallised long-term storage. It unifies two memory concerns that were developed in parallel during v0.8.0: a *deep* tier (episodic and semantic consolidation) and a *surface* tier (spatial telemetry and signature matching), merged into a single memory engine so that

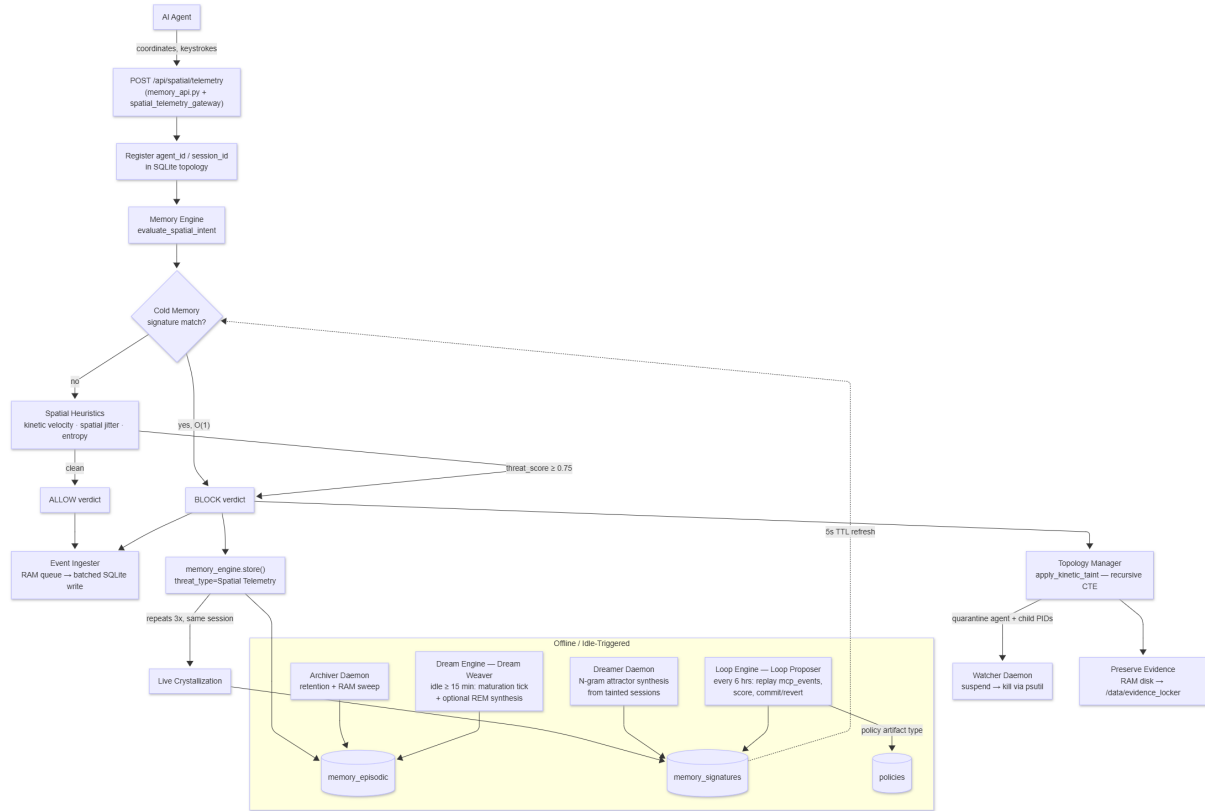


Figure 2: Spatial telemetry and memory data flow. Agent events enter the Memory Gateway, are evaluated against Cold Memory signatures (fast path — no LLM call), and routed through the SpatialHeuristics engine. BLOCK verdicts are enforced directly by the Topology Manager and Watcher Daemon, bypassing the signal loop entirely. ALLOW verdicts continue into the standard pipeline.

telemetry flows through the same episodic/semantic lifecycle as every other verdict rather than maintaining two disconnected substrates.

Table 5: The three-tier ButterClaw memory architecture.

Tier	Storage	Persistence / Transition	Primary Consumer
HOT	In-process working context (e.g. the five most recent successful tool-call events)	Held in RAM for the active session; no durable table of its own	Guardian Brain, Auditor (behavioral drift / <code>timeline_context</code>)
WARM	<code>memory_episodic</code> (SQLite)	Sigmoid <code>activation_strength</code> maturation; records that sustain reinforcement are promoted into the <code>memory_semantic</code> graph by the periodic maturation tick	Dream Weaver, Loop Proposer (pattern recall and consolidation)
COLD	<code>memory_signatures</code> (SQLite)	Crystallises from the semantic graph after sustained repetition within a single session (an identical trajectory recurring 3+ times); provides an O(1) fast-path match; low-confidence entries decay over time	Fleet Sentinel, spatial heuristics fast path (signature matching with no LLM call)

Note on terminology: the standalone `butterclaw-sentinel` SDK (section 9.2) exposes a simplified three-tier mirror of this substrate labelled HOT/EPISODIC/COLD for embedding purposes; the two namings describe the same architectural concern at different levels of fidelity and should not be conflated when citing tier behaviour.

The Dream Weaver and Loop Proposer hemispheres function as offline consolidation processes—they do not participate in the real-time request pipeline and cannot issue BLOCK verdicts. The Dream Weaver’s cycle has two parts: unconditionally, a maturation tick that ages every episodic record’s activation strength, promotes matured entries into the semantic graph, prunes stale ones, and decays low-confidence Cold Memory signatures; and, only when an LLM caller is configured, synthesis of a speculative adversarial scenario from the most-observed semantic-graph entities, written back with `source="dream"` and `verdict="SIMULATED"` so it can never be mistaken for a live verdict. The Loop Proposer reads from COLD memory and the audit log, proposes one signature, policy, or prompt change, replays it against recent `mcp_events` through a side-effect-free shadow evaluator, scores baseline against candidate, and commits or reverts—writing to a cold store pending human review when operating in dry-run mode. Neither hemisphere can modify policy files directly; the human-in-the-loop gate is architectural, not procedural, and the Loop Proposer is explicitly barred from writing to `.py` targets at runtime.

Figure 2 traces this pathway. A second, independent kinetic pathway runs beside the Guardian-Brain-mediated one described above. Agent telemetry events are evaluated first against the COLD Memory signature cache (an O(1) fast path refreshed on a short interval, requiring no LLM call); only on a cache miss does a slower heuristic evaluator score kinetic velocity, spatial jitter, and string entropy against the most recent window of that session’s telemetry. A BLOCK verdict from this fast path is enforced directly—bypassing the Guardian Brain / DRIFT / Paranoia Dial chain entirely—which is why its correctness depends so heavily on the same crystallisation discipline (3+ repeats *within one session*, never pooled across unrelated sessions) that governs ordinary Cold Memory promotion.

5.7 Fleet Detection Stack

Beginning in v0.9.0, ButterClaw extends single-agent containment with a fleet layer that reasons about relationships *between* agents rather than events within one agent’s stream. Three components implement this: a directed, weighted **Trust Graph** recording inter-agent relationships and propagating taint along recorded edges; a **Collusion Detector** that treats collusion roles—`encoder`, `exfiltrator`, `persister`, `scout`, `injector`—as properties of individual *signature hit events* rather than static agent attributes, matched against default complementary pairs (`encoder↔exfiltrator`, `scout↔injector`, `persister↔exfiltrator`, `encoder↔persister`, `scout↔persister`); and the Fleet Sentinel hemisphere (section 5.4) that synthesises both into a fleet-scope verdict.

Fleet state is deliberately isolated from single-agent state: the fleet database and the core runtime database are never co-transacted, so a fault in fleet reasoning cannot corrupt single-agent containment state, and vice versa. A low trust score alone never kills a process or blocks a session directly—it informs the Fleet Sentinel only; kinetic

action still flows through the Guardian Brain, DRIFT, and the Paranoia Dial. This is the architecture’s central fleet-layer commitment: *the Fleet Sentinel perceives; the existing exoskeleton decides*. Taint propagation along the Trust Graph defaults to one hop and requires explicit configuration to extend to two; it is never automated beyond that. Cross-fleet promotion of an entity from session-scope to fleet-scope memory requires a human gate—no automated code path may perform that promotion. The Threat Signature Arsenal remains the single signature system for both individual-agent and collusion detection, so individual and fleet-level pattern matching can never drift out of sync with one another.

5.8 Comparison with Conventional Frameworks

Table 6: ButterClaw vs. Conventional Agentic Frameworks

Dimension	Conventional Frameworks	ButterClaw
Isolation Model	Shared context by default; isolation optional and app-managed	Isolation by default; sharing requires explicit policy authorization
Loop Control	Max iterations at app layer; not architecturally enforced	Hard depth limits at Layer 3; invariant violations trigger automated response
Audit Surface	Logging optional; replay not supported	Append-only chained audit log; full replay capability
Kill Switch	Process termination only; no state preservation; orphans common	Three-tier model with state preservation, drain, and orphan detection; concrete credential destruction (Gibson)
Consent Mechanism	Ad hoc per-application dialogs	Structured consent surfaces per signal class in policy graph; allow-never-short-circuits invariant (I-05)
Multi-Agent Reasoning	Not architecturally distinguished from single-agent reasoning	Dedicated Fleet Sentinel hemisphere and Trust Graph, isolated from single-agent state

6. Kill Switch Architecture

6.1 Why Kill Switches Fail

Existing kill switch implementations fail in three systematic ways:

Async Signal Orphaning: Kill commands terminate the primary agent process but leave asynchronously dispatched signals (to tool APIs, subagents, external services) executing in flight. Those orphaned signals may complete, effect state changes, and spawn further signals after the kill event.

Context Persistence After Termination: Context stored in shared caches or databases survives process termination. A subsequent agent startup may inherit contaminated context, effectively restarting the killed signal chain.

Cascading Multi-Agent Signals: Kill commands issued to one node in a multi-agent graph do not automatically propagate to downstream agents that have already received derived signals.

A kill switch that terminates the process but does not resolve the signal graph is not a kill switch. It is a process manager. The distinction is the difference between a system that stops and one that continues mutating state under no active oversight.

6.2 The Three-Tier Kill Switch Model

Tier 1 — Signal Pause: Suspends all signal propagation. In-flight signals paused at current routing hop; full state checkpointed. *Reversible*. Appropriate for non-urgent anomalies requiring human review.

Tier 2 — Loop Drain: Blocks new initiations; allows in-flight signals to complete to their nearest resolution checkpoint. Signals that do not resolve at that checkpoint are paused and do not re-enter the routing loop. Appropriate for planned wind-downs or moderate anomalies.

Tier 3 — Hard Termination: Immediately voids all active signals. State snapshot saved to forensic store; forensic store locked. All signal identifiers written to orphan watchlist. Subsequent callbacks matching those identifiers quarantined. Appropriate for confirmed harmful behavior, security incidents, or regulatory directives. In the reference implementation, Tier 3 is the point at which the Gibson credential-destruction sequence (section 6.6) is triggered.

6.3 Kill Switch Triggers

Manual: Authorized human operator selects tier via operations interface.

Automated: Loop Monitor invariant violation exceeds severity threshold; tier determined by severity configuration. In the reference implementation this is realised by the Paranoia Dial: Level 2 terminates the offending process, and Level 3 triggers Gibson and system lockdown.

Federated: Peer agent or governance layer issues cryptographically attested cross-system kill command; verified by receiving system’s routing engine before execution.

6.4 The Orphan Signal Problem

Before any signal is dispatched to an external system, its identifier is registered in the orphan watch registry. After a kill event, the registry is queried continuously; any callback, webhook, or return signal matching a registered identifier is intercepted at Layer 0, classified as an orphan, quarantined, logged, and presented to human review.

6.5 Recovery and Re-Entry Protocols

Re-entry after a kill event requires: (1) resolution of all orphan activity; (2) context membrane flush and re-initialization; (3) audit log review and root cause determination; (4) routing policy review and update for the signal class that caused the violation.

Table 7: Kill Switch Tier Summary

Tier	Trigger	Signal Fate	State Fate	Recovery
1: Pause	Non-urgent anomaly; human review needed	Suspended at hop; state preserved	Checkpoint written; memory retained	Resume from checkpoint after clearance
2: Loop Drain	Planned wind-down; moderate anomaly	Complete current cycle; no new init	Resolved closed; unresolved paused	Fresh init after quiesce & policy review
3: Hard Terminate	Confirmed harm; security incident; directive	Immediately voided; orphan watchlist	Forensic snapshot; store locked; credential vault destroyed (Gibson)	Full RCA; membrane flush; re-authorization

6.6 Gibson: Concrete Credential Wipe Implementation

Where section 6 above describes the kill switch in the abstract, the reference implementation’s Tier 3 response includes a concrete, named mechanism—**Gibson**—for destroying credential material so that a compromised or misbehaving deployment cannot continue to authenticate after termination. Gibson is triggered by a Paranoia Level 3 verdict or by a manual administrator call, and it executes as three atomic steps: (1) every ciphertext row in the credential vault is overwritten with cryptographic garbage, encrypted under a freshly generated, immediately discarded Fernet “poison” key; (2) every HMAC-SHA256 API key hash is deleted; and (3) the in-memory session-signing key cache is invalidated. If a hardcoded, non-configurable `DRY_RUN` flag is set, the sequence returns immediately without touching any of the three—a pattern deliberately mirrored by the Dream Weaver’s own non-overridable dry-run flag (section 5.6), because both guard against an irreversible action being taken by a process that was never meant to take one.

Two design choices make Gibson something more than an undifferentiated wipe. First, because session tokens are HMAC-signed with a key derived from the vault’s master key, destroying the vault automatically renders every active session cryptographically unverifiable—there is no separate “revoke all sessions” step to forget. Second, and deliberately asymmetric: *policy rules and accumulated memory survive Gibson by design*. Wiping containment policy or learned behavioral memory during incident response would leave the system defenseless at the moment of re-authorization, so the credential wipe and the preservation of policy/memory are two expressions of the same underlying principle—an incident should strip an attacker’s standing access without also stripping the defender’s accumulated containment knowledge.

7. The Signal Containment Model

7.1 Containment as a First-Class Design Primitive

In ButterClaw, containment is defined *before* the functional architecture. The routing engine, context membrane, loop monitor, and kill switch are not additions to the signal processing system—they *are* the signal processing system. Functionality is implemented within the constraints they define, not despite them. This inversion is the central architectural commitment of ButterClaw, and it is not achievable by retrofitting containment onto an existing unconstrained agentic framework.

7.2 The Four Containment Primitives

Definition 7.1 — The Four Containment Primitives

Boundary: A signal cannot cross defined perimeters without explicit authorization, enforced jointly by the routing engine and context membrane.

Depth: Recursion/loop depth is hard-bounded; signals at the limit route to an escalation queue, not to further recursion.

Consent: Any action modifying external state requires a verified consent authorization, recorded in the provenance and audit log.

Observability: Every signal event is logged to the append-only audit store in real time. Audit store unavailability triggers system-wide Tier 1 kill; it is a precondition for operation.

7.3 Formal Properties

Safety

For every external state change s effected by the system, there exists a consent authorization record c in the audit log that precedes s and explicitly authorizes the action class of s .

Liveness

For every signal σ with initiation time t_0 , there exists a terminal event (resolution, escalation, or kill) before $t_0 + T_{\max}$, where $T_{\max}$ is the configured maximum signal lifetime.

Fairness

For every signal class C and every time window W , signals of class C that enter during W receive routing attention proportional to their configured priority weight.

7.4 Containment Threat Matrix

7.5 Implementation Mapping

The four containment primitives above are deliberately abstract so that they generalise beyond any one implementation. Table 9 maps them onto the specific, numbered invariants enforced in the v0.9.2 reference implementation—the concrete claims an auditor could check against the running system rather than against the architecture alone.

Table 8: Containment Threat Matrix

Threat	Primitive Violated	ButterClaw Mitigation
Signal crosses domain boundary without authorization	Boundary	Routing engine policy graph blocks; signal quarantined & alert issued
Recursive subagent spawning causes loop divergence	Depth	Hard depth limit; signal at limit routed to escalation queue; automated Tier 1 kill
Signal modifies external state without human awareness	Consent	Consent surface required; policy must explicitly authorize silent consent; audit preserved
Signal executes without audit log entry	Observability	Audit store unavailability triggers system-wide Tier 1 kill
Adversarial prompt injected into signal payload	Boundary + Obs.	Context membrane payload auth detects modification; Layer 0 adversarial classifier; quarantine
Loop amplification attack	Depth + Obs.	Signal initiation rate tracking; rapid initiation triggers Tier 2 kill; depth hard limit
Context contamination across signal boundaries	Boundary	Context membrane enforces per-signal scoping; cross-signal sharing requires explicit auth event

Table 9: Containment primitives mapped to enforced implementation invariants (v0.9.2).

Primitive	Invariant(s)	Concrete Guarantee
Boundary	I-01, I-02, I-06, I-07-fleet	The vault master key exists only in the OS keyring and is never written to disk, env vars, or logs (I-01); all stored credentials pass through Fernet encryption before being written, so a compromised database file without the master key yields only ciphertext (I-02); only one watcher instance may run per host, enforced by a PID lock (I-06); the fleet database and the core runtime database are never co-transacted, so a fault in one cannot corrupt the other (I-07-fleet).
Depth	I-07, I-08, I-05-fleet	ChainExecutor enforces a hard ceiling of ten steps per chain under a sixty-second total timeout (I-07); the watcher retry queue is capped at 100 entries, with excess silently dropped rather than unbounded (I-08); cross-agent taint propagation defaults to one hop and requires explicit configuration, never automated, to extend to two (I-05-fleet).
Consent	I-05, I-02-fleet, I-04-fleet	An ALLOW policy verdict is logged but never short-circuits evaluation of subsequent rules in the same scope (I-05, section 5.5); a trust score of zero does not itself kill a process or block a session—it only informs the Fleet Sentinel, and kinetic action still requires the Guardian Brain + DRIFT + Paranoia Dial chain (I-02-fleet, I-04-fleet).

continued on next page

Table 9 (continued)

Primitive	Invariant(s)	Concrete Guarantee
Observability	I-03, I-04, I-09, I-10	The Gibson credential-wipe sequence executes as three explicitly ordered, auditable steps rather than an undifferentiated wipe (I-03, section 6.6); session-signing keys derive from the vault master key, so a wiped vault deterministically invalidates every session rather than leaving ambiguous state (I-04); the log sanitizer is a targeted blacklist, not an aggressive whitelist, preserving enough structure for the Guardian Brain to evaluate full prompt-injection attempts rather than silently discarding evidence of them (I-09); inbound transport reads are bounded at the byte level before the JSON parser engages, preventing an out-of-memory crash from silently destroying audit evidence (I-10).

8. Related Work

8.1 Agent Frameworks

The conventional-frameworks comparison in Table 6 is deliberately framed against a generic baseline rather than any single project, because containment posture is uneven across the ecosystem rather than uniformly absent. Composability-first frameworks such as LangChain [6] and reasoning-and-acting formulations such as ReAct [25] treat iteration limits, tool-call scoping, and audit logging as application-layer concerns left to the integrator, not as architectural guarantees the framework itself enforces. Multi-agent orchestration frameworks such as AutoGen [24] and role-based software-engineering agent societies such as MetaGPT [11] extend this same pattern to inter-agent communication: they standardise *how* agents exchange messages and delegate subtasks, but leave isolation, consent, and kill-switch semantics to the surrounding deployment. Generative-agent simulations of believable human behavior [17] and self-reflective agents that revise their own trajectories based on verbal feedback [21] further illustrate the trend this paper argues against: capability and autonomy have grown fastest in exactly the dimensions (memory, self-revision, multi-agent delegation) that the signal model frame identifies as the new safety surface, without a corresponding growth in architecturally-enforced containment.

8.2 Prompt Injection

The context membrane (section 5) and the DRIFT policy engine’s payload scanning (section 5.5) are ButterClaw’s responses to a threat class documented extensively in prior work. Indirect prompt injection via untrusted retrieved content was characterised by [8] as a structural consequence of LLM applications that cannot distinguish instructions from data at the point of ingestion; earlier adversarial prompting work [18] and the broader survey of injection attacks and defenses [13] establish that filtering at the output stage—the conventional “language model” response—does not address injection that occurs upstream, at the point where untrusted content enters the context window. This paper’s signal model framing reframes the same threat as a *boundary* violation (Definition 7.1) enforced at ingestion (Layer 1) rather than a content-moderation problem addressed after the fact.

8.3 Kill Switch Research

The theoretical foundation for corrigibility and safe interruptibility predates this paper’s architecture by nearly a decade. The off-switch game [10] formalises the incentive problem—an agent optimising a fixed objective has no inherent reason to tolerate being switched off, and corrigibility must be designed in rather than assumed. Safely interruptible agents [16] and the technical research agenda for aligning superintelligence [22] likewise treat interruptibility as a property that must survive the agent’s own learning process, not merely a process-management feature bolted on afterward. ButterClaw’s three-tier kill switch model and its orphan-signal watch registry (section 6) are an attempt to answer, at the systems-engineering level, the same question this literature poses at the theoretical level: what does it mean for a termination signal to actually terminate, rather than merely appearing to?

9. Software Artifact and Reproducibility

9.1 Repository

The ButterClaw runtime described in this paper is released as open-source software under the Apache-2.0 license at <https://github.com/butterclaw-tech/butterclaw>. The release accompanying this whitepaper revision corresponds to runtime version **v0.9.2**, which introduced the Fleet Detection Stack (section 5.7) and the fifth reasoning hemisphere on top of the v0.8.0 memory and offline-consolidation substrate (section 5.6).

9.2 The butterclaw-sentinel SDK

Alongside the full runtime, a dependency-free, embeddable Python library— **butterclaw-sentinel**—is released at <https://github.com/butterclaw-tech/butterclaw-sdk>. The SDK ports seven internal subsystems directly from the v0.9.2 source behind a single **Sentinel** facade: **SentinelCore** (the three background daemons—fallback ticker, dream loop, loop engine), **AgentRegistry**, **PolicyEngine** (the DRIFT evaluator of section 5.5), **TrustGraph**, **DriftDetector** (a generalisation of the spatial-telemetry heuristics of section 5.6 to arbitrary numeric metrics, hardcoded to the v0.9.2 thresholds `MAX_TYPING_VELOCITY=15.0` and a spatial-jitter threshold of 800), **CollusionDetector**, and **MemorySubstrate** (the SDK’s HOT/EPISODIC/COLD mirror of the runtime’s HOT/WARM/COLD architecture; see the note in section 5.6). The SDK exists to let integrators embed ButterClaw’s containment primitives—particularly the DRIFT evaluator and its I-05 invariant— into a host application without running the full Flask server, and it carries the same core invariants (I-13, I-14, I-15) forward into its own design-invariant register.

9.3 Citation

Researchers and practitioners building on this work should cite both the whitepaper (via its Zenodo DOI, minted for this record) and, where the runtime itself is used, the software repository directly:

Listing 2: Suggested citation form

Bc77. "Autonomous Signals: A New Ontology for AI Agents."
 ButterClaw Tech, 2026. Zenodo. <DOI to be inserted on publication>.
 Software: <https://github.com/butterclaw-tech/butterclaw> (v0.9.2)
 SDK: <https://github.com/butterclaw-tech/butterclaw-sdk>

9.4 Version Binding

This revision of the whitepaper is bound to runtime version **v0.9.2**. Readers should note that the Zenodo record’s own version field tracks the *whitepaper*’s revision history (currently **v1.0.0** at time of writing) independently of the *software*’s semantic version—the two are related but distinct, and a later whitepaper revision describing a later runtime version will increment the Zenodo version without implying a corresponding software major-version bump, or vice versa. Claims in Sections 5–7 that cite specific temperatures, invariant IDs, or table/column names should be understood as accurate as of v0.9.2 and re-verified against **ARCHITECTURE.md** before being relied upon for a different runtime version.

9.5 Practical Deployment

The audit store’s availability-as-prerequisite constraint demands that enterprises treat audit infrastructure with the same reliability requirements as their primary data tier. Consent surface design requires organizational policy work before technical implementation: which action classes require interactive human consent, which are eligible for policy pre-authorization, and which are never authorized regardless of instruction. This is a governance exercise, not an engineering one.

9.6 Interoperability

ButterClaw wraps existing LLM APIs rather than replacing them. Model inference calls are signal transformations executed within Layer 2; the API call is not modified. Existing agent frameworks can be adapted by implementing adapters that translate framework-native constructs into ButterClaw signal events before execution—the SDK’s own three-method adapter contract (**connect/adapt/sync**, section 9.2) is one concrete instance of this pattern. Frameworks with opaque execution loops require deeper instrumentation or replacement with ButterClaw-native components.

10. Open Problems

Formal Verification of Loop Convergence: Can convergence guarantees be provided for signal loops containing learned model components? Recent work in neural network verification and abstract interpretation provides partial tools, but the compositional complexity of multi-step agentic loops with non-deterministic routing remains an open formal methods problem.

Cross-Organizational Federated Kill Switches: Kill switch coordination across organizational boundaries becomes a multi-party protocol design problem. What consensus mechanism governs cross-org kill events? What liability framework governs actions taken on the basis of a federated kill signal? These are simultaneously technical, legal, and governance questions.

Signal-Level Interpretability: Current interpretability research focuses on individual model inference internals. Signal-level interpretability asks why the system routed a signal a particular way, and what accumulated context at each hop reveals about effective system objectives—requiring new tools operating at the signal graph level.

Automatic Containment Policy Generation: Future work should explore whether containment policies can be learned from operator demonstrations, with depth limits inferred from task complexity distributions and consent surfaces adapted dynamically from signal behavior patterns.

11. Conclusion

This paper has argued for a fundamental ontological shift: from the language model frame—in which the system is a text predictor and safety is a content property—to the signal model frame, in which the system is a signal transducer and safety is an architectural property defined at signal boundaries, depths, consent surfaces, and observability.

“We cannot engineer defenses against failure modes we cannot name. The signal model frame makes the safety surface of autonomous AI systems legible for the first time.”

ButterClaw is our proposal for what containment-first architecture looks like when signal model thinking is taken seriously. Its six layers, five reasoning hemispheres, named DRIFT policy engine, three-tier memory substrate, fleet detection stack, three kill switch tiers (with a concrete credential-destruction mechanism), four containment primitives, and formal properties are offered as a concrete engineering target—one we have mapped, invariant by invariant, onto a running v0.9.2 implementation rather than leaving as architecture diagrams alone. We expect and welcome disagreement about specific design choices. What we ask is that disagreement be conducted at the signal level, not the token level.

The research and engineering community has built extraordinary systems. The next task is to contain them—not to diminish their capability, but to ensure that capability is exercised within boundaries that are legible, auditable, and subject to human control.

References

- [1] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané. Concrete problems in AI safety. *arXiv:1606.06565*, 2016.
- [2] Anthropic. Claude’s model specification. *Anthropic Technical Documentation*, 2024.
- [3] Y. Bai, A. Jones, K. Ndousse, et al. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv:2204.05862*, 2022.
- [4] R. Bommasani, D. A. Hudson, E. Adeli, et al. On the opportunities and risks of foundation models. *Stanford CRFM Technical Report*, 2021.
- [5] M. Brundage, S. Avin, J. Clark, et al. The malicious use of artificial intelligence: Forecasting, prevention, and mitigation. *Future of Humanity Institute Technical Report*, 2018.
- [6] H. Chase. LangChain: Building applications with LLMs through composability, 2022–2024. <https://github.com/langchain-ai/langchain>. Accessed October 2026; repository has no single pinned release corresponding to the framework behavior discussed here.
- [7] P. Christiano, J. Leike, T. Brown, et al. Deep reinforcement learning from human preferences. In *NeurIPS 2017*.
- [8] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023.

- [9] D. Hadfield-Menell, S. Milli, P. Abbeel, S. Russell, and A. Dragan. Inverse reward design. In *NeurIPS 2017*.
- [10] D. Hadfield-Menell, A. Dragan, P. Abbeel, and S. Russell. The off-switch game. In *Proceedings of the 26th International Joint Conference on Artificial Intelligence (IJCAI)*, 2017.
- [11] S. Hong, X. Zheng, J. Chen, et al. MetaGPT: Meta programming for a multi-agent collaborative framework. *arXiv:2308.00352*, 2023.
- [12] J. Leike, M. Martic, V. Krakovna, et al. AI safety gridworlds. *arXiv:1711.09883*, 2017.
- [13] Y. Liu, G. Deng, Y. Li, et al. Prompt injection attacks and defenses in LLM-integrated applications. *arXiv:2310.12815*, 2023.
- [14] M. Minsky. *The Society of Mind*. Simon and Schuster, 1986.
- [15] OpenAI. GPT-4 technical report. *arXiv:2303.08774*, 2023.
- [16] L. Orseau and S. Armstrong. Safely interruptible agents. In *Proceedings of the 32nd Conference on Uncertainty in Artificial Intelligence (UAI)*, 2016.
- [17] J. S. Park, J. C. O’Brien, C. J. Cai, et al. Generative agents: Interactive simulacra of human behavior. In *ACM UIST 2023*.
- [18] F. Perez and I. Ribeiro. Ignore previous prompt: Attack techniques for language models. *arXiv:2211.09527*, 2022.
- [19] S. Russell. *Human Compatible: Artificial Intelligence and the Problem of Control*. Viking Press, 2019.
- [20] Y. Shavit. What does it take to catch a chinchilla? verifying rules on large-scale neural network training runs. *arXiv:2303.11341*, 2023.
- [21] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS 2023*.
- [22] N. Soares and B. Fallenstein. Aligning superintelligence with human interests: A technical research agenda. *MIRI Technical Report 2014-8*, 2014.
- [23] L. Weng. LLM-powered autonomous agents. *Lilian Weng’s Blog, OpenAI*, 2023. <https://lilianweng.github.io/posts/2023-06-23-agent/>.
- [24] Q. Wu, G. Bansal, J. Zhang, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv:2308.08155*, 2023.
- [25] S. Yao, J. Zhao, D. Yu, et al. ReAct: Synergizing reasoning and acting in language models. In *ICLR 2023*.
- [26] D. M. Ziegler, N. Stiennon, J. Wu, et al. Fine-tuning language models from human preferences. *arXiv:1909.08593*, 2019.